

"LECTURE # 06"

CIRCULAR QUEUE:

A circular queue is a FIFO (First In, First Out) data structure where the last node connects back to the first node, forming a circle.

- 1) It means ka han jab 1 node banain ga to front aur rear ussi ko point karin ga aur un ka next ma bhi same hi ho gi.
- 2) Jab han 2nd node banain ga to rear uss newnode ko point kary ga jab ka front same rahay ga. Lakin rear ka next ma ab front aa jay ga aur front ka next ma ab wo newnode ho gi.
- 3) Jab han 3rd node banain ga to rear again new node ko point kary ga jabka front same rahay ga. rear ka next ma front hi rahay ga aur front ka next ma 2nd node hi rahay gi, bas 2nd node ka next ma newnode aa jay gi aur issi tarah continue rahay ga.

* Why Circular:

-) We can reuse memory efficiently in array form.
-) And rotation becomes very easy.

* Key Operations:

① enqueue():

Insert element at the end (rear).

Jo node bhi enqueue karin ga wo end se hogi aur rear uski ko point karay (sab se last entry ko).

② dequeue():

Delete element from the front.

Hamesha front pa jo node ho gi wohi dequeue hogi aur front apni sa baad wali node ko point karay ga.

③ peek():

peek karay se queue ka front wala element/node show ho ga.

④ display()

⑤ isempty():

⑥ isfull():

ho ga. Ya sirf array based ma

⑦ frontelement():

front ka data show karna.

⑧ rear element():

last (rear) dekna.

★ Structure of a Node:

```
class Node
```

```
{
```

```
    public:
```

```
        int data;
```

```
        Node* next;
```

```
        Node(int v): data(v), next(NULL){}
```

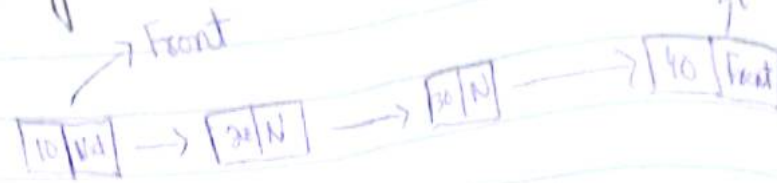
```
};
```

1) Ya node class ha, iss ma data aur sirf next ho ga.

2) next as usual linke karney ka liay hota he.

3) data bhi as usual.

* Diagram :



* Structure of a Queue Class:

```
class Queue
```

```
{
```

```
    Private:
```

```
        Node* front;
```

```
        Node* rear;
```

```
    Public:
```

```
        Queue(): front(NULL), rear(NULL) {}
```

```
        functions();
```

```
};
```

•) front points to the first node in the queue, it is always the first one.

•) rear always points to the last entry in the queue

•) In public there will be a constructor & remaining functions.

//Code:

```
#include <iostream>
using namespace std;
```

```
class Node
```

```
{
```

```
public:
```

```
int data;
```

```
Node* next;
```

```
Node(int v): data(v), next(NULL) {}
```

```
};
```

```
class Queue
```

```
{
```

```
private:
```

```
Node* front;
```

```
Node* rear;
```

```
public:
```

```
Queue() : front(NULL), rear(NULL) {}
```

```
bool isempty()
```

```
{
```

```
return (front == NULL);
```

```
}
```

```
// for node
```

```
// bool isempty()
```

```
// {
```

```
// return (front == -1);
```

```
// }
```

```
// for array based
```

```
// bool isfull()
// {
//     return ((rear+1)%size == front);
// }
```

// for array based

void enqueue(int value)

```
{
    Node* newnode = new Node(value);
```

```
    if (isempty())
```

```
    {
        front = rear = newnode;
```

```
    }
```

```
    rear->next = front;
```

```
    }
```

```
    else
```

```
    {
```

```
        rear->next = newnode;
```

```
        rear = newnode;
```

```
        rear->next = front;
```

```
    }
```

```
}
```

// for node based

// agar empty ho to front aur rear same node ko point karin ga. aur rear ka next ma front aay ga for circular moment, as both pointing to the same node so both's next will be same too.

// Agar pehlay ki koi node ho to new node ki entry ka waqt ~~rear~~ ka next ma newnode dalin ga so front ka next ma bhi newnode ajay gi because both front & ~~rear~~ are pointing to the same node.

// then ham ~~rear~~ ko update karin ga as ~~rear~~ should always be pointing to latest entry so $\text{rear} = \text{newnode}$ karin ga.

// then for mainting circularity, again ~~rear~~ ka next ma front dal dain ga.

```
void enqueue(int v)
{
    if (isfull()) { return; }

    if (front == -1) { front = 0; }
    {
        rear = (rear + 1) % size;
        arr[rear] = v;
    }
}
```

// for array based

// if 1st node, update front to 0

// updating rear to less than size or ~~size~~ ^{again} to zero is dequeue happened.

```

void dequeue()
{
    if (isEmpty()) { return; }

    if (front == rear)
    {
        delete front;
        front = rear = NULL;
    }
    else
    {
        Node* temp = front;
        front = front->next;
        rear->next = front;
        delete temp;
    }
}
// for node based

```

```

void deque()
{
    if (isEmpty()) { return; }

    if (front == rear)
    {
        front = rear = -1;
    }
    else
    {
        front = (front + 1) % size;
    }
}
// for array based

```

// update front, like if front was at 0 index when it was updated like $\text{front} = (\text{front} + 1) \% \text{size}$ it will run like $\text{front} = (0 + 1) \% 5$ (supposed), it will become $1 \% 5$ which is 1, so front shifts to next position.

```
void display()
```

```
{  
    if (isEmpty()) { return; }
```

```
    int i = front;
```

```
    while (true)  
    {
```

```
        cout << arr[i];
```

```
        if (i == rear)
```

```
        { break; }
```

```
        i = (i + 1) % size;
```

```
    }
```

```
}
```

// for array based.

// $i = (i + 1) \% \text{size}$, we are doing it to move i to the next index,

$i = (0 + 1) \% \text{size} = 1$

$i = (1 + 1) \% \text{size} = 2$

$i = (2 + 1) \% \text{size} = 3$ and so on.

Queue (All Questions)

(Question #01)

class Queue

{
private;

int *ary;
int front;
int rear;
int cap;
int cnt;

// Data members
// first element
// last element
// tot capacity

public;

Queue(int s)

{

cap = s;
ary = new int[cap];
front = 0;
rear = -1;
cnt = 0;

}

// Constructor

// Initialization

~Queue()

{

delete [] ary;

}

// destructor

bool isep() { return cnt == cap; } // if no entry

bool isfull() { return cnt == cap; } // if full

```
void enqueue(int x)
{
```

```
    if (isfull())
```

```
    {
        return;
    }
```

```
    rear = (rear + 1) % cap;
```

```
    arr[rear] = x;
```

```
    cnt++;
}
```

// if cap = full

0% suppose
rear = 0

→ // arr[0] = 10

→ cnt = 1

```
void dequeue()
{
```

```
    if (isempty())
```

```
    {
        return;
    }
```

// if nothing (cnt = 0)

```
    front = (front + 1) % cap;
```

```
    cnt--;
```

```
}
```

$1 \% 4 = 1$

→ cnt = 0

```
void display()
{
```

```
    if (isempty())
```

```
    {
        return;
    }
```

```
    for (int i = 0; i < cnt; i++)
```

```
    {
        int ind = (front + i) % cap;
```

```
        arr[ind];
    }
```

let $f = 0$

```
        int ind = (front + i) % cap;
```

$0 \% 2 = 0$
ind = 0

then $1 \% 2 = 1$

then $2 \% 2 = 0$ if $i < cnt$

Question # 02

explained in basic operations of Queue

DRY_RUN

Suppose: User enters 5

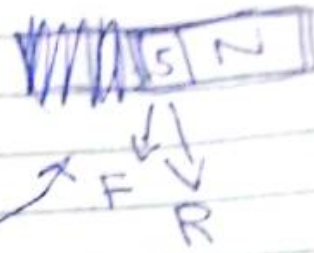
Node* nn = new Node(5);

if (→) ✓

F = R = nn;

return;

}



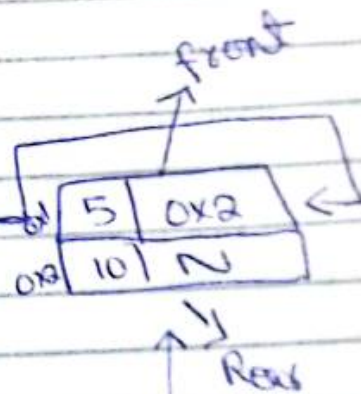
Now enqueue 10:

Node* nn = new N(10);

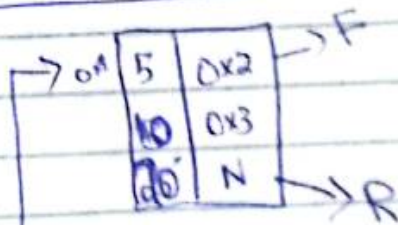
if (→) ✗

rear->next = nn;

rear = nn;



Now 20:



Now dequeue:

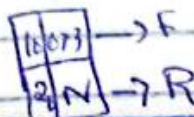
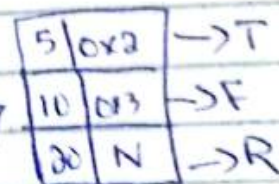
if (→) ✗

Node* t = F;

F = F->next;

if (F == Null) ✗

delete temp;



Question # 03

class PriorityQueue

{
private: arr;
int data;
int priority;
int cap;
int size;

// Data members

public:
PriorityQueue(int c)

// constructor

{
cap = c;
arr = new int[cap];
priority = new int[cap];
size = 0;

// initialized

~PriorityQueue()

// destructor

{
delete [] arr;
delete [] priority;
}

void enqueue(int v, int p)

{
if (size == cap) { return; }

// full

arr[size] = v;
priority[size] = p;
size++;

// insertions

// increment size

}

topnode()

```
if (size == 0) { return; }
```

// no element

```
int hp = 0;
```

```
for (int i = 1; i < size; i++)
```

```
{ if (priority[i] > priority[hp])
```

```
    hp = i;
```

// finding element
with highest
priority
index found here

```
for (int i = hp; i < size - 1; i++)
```

```
    arr[i] = arr[i+1];
```

```
    priority = priority[i+1];
```

Shifting 1
step before
to maintain
linearity

```
size--;
```

// decrement size

void front()

```
if (size == 0) { return; }
```

// if no element

```
int hp = 0;
```

```
for (int i = 1; i < size; i++)
```

```
{ if (priority[i] > priority[hp])
```

```
    { hp = i; }
```

// finding element
with highest
priority

Question # 04

// same codes for Stack & Queue
(DRY - Run)

void reverse (Queue q, int k)
{
if (k <= 0 || k > q.size()) return;
Stack S;

k=2

Suppose:

1	-
2	-
3	-
4	N

1) for (int i=0; i<k; i++)
{
q.push(q.dequeue());
}

2	-
1	-

2) while (!s.empty())
{
q.enqueue(s.pop());
}

3	-
4	-
2	-
1	-

3) int remaining = q.size() - k;

4-2=2

4) for (int i=0; i<remaining; i++)
{
q.enqueue(q.dequeue());
}

3	-
4	-
2	-
1	-

Question # 05

same code of Stack & Queue
(DRY_RUN)

bool ispalindrome(queue & q)

if (q.empty())

Suppose

1	
2	
1	

Stack S;

Queue T;

int qs = q.size(); $qs = 3$

for (int i = 0; i < qs; i++) ✓

int val = q.dequeue();

S.push(val);

T.enqueue(val);

q.enqueue(val);

}

val = 1

S → [1]

T → [1]

2	
1	

2	
1	
1	

for (—) ✓

int v = q.de();

S.push(v);

T.enq(v);

q.enq(v);

}

val = 2

1	
1	

2	
1	

1	
2	

1	
1	
2	

o) again $foo(\rightarrow) \checkmark$ $2 < 3$

int $v = q \cdot degl()$ $v = 1$

s.push(v);

1	
2	
1	

T.enqueue(v);

1	
2	
1	

q.enqueue(v);

1
2
1

o) bool ispanz-tree;

while (!T.isep()) $\checkmark$

{
if (T.deg() != s.pop()) $\times$

o) wh($\rightarrow$) $\checkmark$

{

if() $\times$

o) wh() $\checkmark$

{

if($\rightarrow$) $\times$

o) wh($\rightarrow$) $\times$

return topalg